PROJECT 02 LAM · TEAM OF 5

Search assistant for space-mission knowledge

Build a question-answering assistant over LAM's mission documentation and data servers that always shows its sources and says 'I don't know' when the answer is not there.

The context in brief

LAM's knowledge about major space missions is spread across several systems: WordPress web pages, the Redmine project tracker (issues, wikis), and two data servers called Atrium and Baggera (spelling to confirm) that host astrophysical data and documents. Finding the right, up-to-date information takes time, especially for newcomers.

Why it matters. Faster access to reliable knowledge, easier onboarding and better preservation of the laboratory's memory. In science, a fluent answer without a source is useless: traceability is the key requirement.

What you receive

- Access (or exports) for a selection of sources: WordPress, Redmine, Atrium, Baggera.
- Questions from LAM engineers and researchers, with the expected answers or documents.
- A LAM contact for each source and one hour of follow-up per week.

What you deliver

- A RAG prototype on one mission and two or three sources, with cited answers and a web interface.
- Connectors and a parsing pipeline that keep source, date, version and access rights.
- A test set of 50–100 questions and an evaluation against a keyword-search baseline.
- A short recommendation for extending to more sources and missions.

Suggested team organisation (5 students)

- **Connectors (2 students):** WordPress, Redmine, Atrium and Baggera access, parsing, metadata.
- **Retrieval (1):** chunking, BM25, embeddings, hybrid search, reranking.
- **Answer & interface (1):** prompting with citations, abstention, web UI.
- **Evaluation & LAM liaison (1):** question set, metrics, user tests, coordination.

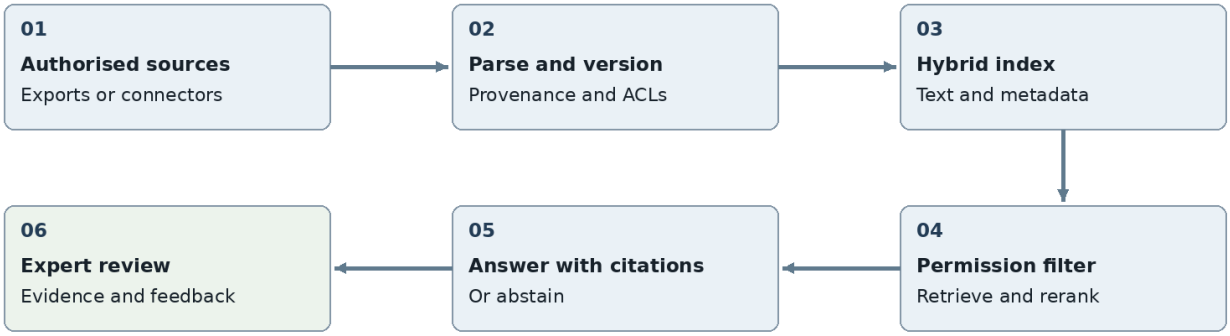
Questions to ask at the kickoff meeting

- What are the exact names and roles of Atrium and Baggera, and how can we access them? Which mission and sources first?
- Who may see which documents, and can you give us 50 real questions with their expected answers?

PROJECT 02 LAM

How to start: a practical method

Follow these steps in order. Build the simplest version end to end first, measure it, then improve the weakest step. Tools are suggestions: choose them with your supervisor and keep everything open source and local when the partner asks for sovereignty.



Target pipeline. Each box is one module you can build and test separately.

Step	What you do	Suggested tools (open source)
1	List each source: what it contains, how to access it (API, export, file listing), volume, formats, who may see what.	spreadsheet; WordPress REST API (/wp-json/wp/v2/…), Redmine REST API (issues.json, wiki)
2	Choose one mission and 2–3 sources; take a dated snapshot of the content.	Python requests, rsync or server exports
3	Convert everything to one common format: id, source, url, title, date, version, text, access group. For science files, index the metadata (headers, descriptions), not the raw data.	trafilatura, BeautifulSoup, PyMuPDF, Docling, astropy (FITS headers)
4	Cut documents into passages that follow their structure (sections, one issue + its comments), keep the title path.	LangChain or LlamaIndex text splitters, or your own code
5	Baseline: keyword search (BM25). Then embeddings, then hybrid search (fusion of both), then optional reranker.	rank_bm25 or OpenSearch, sentence-transformers, Qdrant, cross-encoder
6	Generate the answer only from retrieved passages, with citations and links; answer 'not found in the sources' when evidence is weak.	Ollama + open-weights LLM
7	Filter by access rights before retrieval; handle updates and deletions.	metadata filters in the vector store
8	Evaluate with the question set, including questions with no answer.	pandas, Ragas (optional)

Core logic (pseudo-code)

```
docs = []
for source in [wordpress, redmine, atrium, baggera]:   # selected subset
    for item in source.export(mission='X'):
        docs += to_units(item)           # id, url, date, version, text, groups
index = HybridIndex(bm25=True, embeddings=model).build(chunk(docs))

def answer(question, user):
    hits = index.search(question, k=8, allowed=user.groups)
    if not hits or hits[0].score < MIN_SCORE:
        return 'Not found in the indexed sources.'
    return llm(answer_prompt, question, hits)    # cites [doc id, date]
```

Spelling to confirm. Atrium and Baggera are LAM data servers hosting astrophysical data and documents. Confirm their exact names, access methods and content at the kickoff meeting.

Data, not just text. Data servers may hold scientific files (images, tables, binary formats). In nine weeks, index their descriptions and metadata, and link to the original file rather than trying to read the data itself.

Week-by-week plan



Week	Goal of the week	You are done when…
1	Meet LAM users; inventory WordPress, Redmine, Atrium and Baggera; choose one mission and 2–3 sources.	Source sheet and access are ready.
2	Snapshot the content; collect 50–100 expert questions (with 10 unanswerable ones); keyword baseline.	Questions are frozen and the baseline is scored.
3	Parse and chunk into the common format with metadata and versions.	Each passage links back to the right source.
4	Compare BM25, embeddings and hybrid search (+ reranker).	The best retrieval setting is chosen with numbers.
5	Cited answers, abstention and a simple web interface.	Every displayed fact can be checked.
6	Access rights, updates and deletions.	No restricted document appears to the wrong user.
7	Evaluation with LAM users on unseen questions.	Results and failure analysis are ready.
8	Freeze: repository, connectors configuration, demo video.	Tagged release delivered to LAM.
9	Report, slides, rehearsal of the defence.	No new development.

How you will know it works

What you measure	How	Starting target*
Finds the right passage	Recall@5: the expected passage is in the top 5	≥ 85 %
Answers are supported	Share of statements backed by the cited passages	No unsupported key fact
Knows when to stop	Unanswerable questions correctly declined	≥ 80 %
Better than keywords	Compare with BM25 alone on the same questions	Clear improvement
Speed	Time to answer a question	< 10 s on the agreed machine

** Targets are starting proposals to agree with the partner in week 1, not promises.*

Pitfalls to avoid

- Do not try to index everything: one mission done well beats ten done badly.
- Keep the date and version of each document, or old specifications will look current.
- Do not show the title of a document the user is not allowed to see, even in an error message.
- Ask experts for hard questions (identifiers, acronyms, several documents), not only easy ones.